



TCC805x Automotive Security Solution

User Guide for HSM

**Rev. 1.10 [G]
2020-08-05**

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

© 2020 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
1.1	Motivation.....	3
1.2	Feature.....	3
1.3	Abbreviation.....	4
2	Use Cases	5
2.1	Flashing Secure	5
2.2	Secure Communication.....	5
2.3	Feature Activation.....	5
3	How to Use HSM.....	6
3.1	HSM Driver for Linux on Cortex-A72 and Cortex-A53	6
3.1.1	Name of Driver	7
4	How to Use ioctl on Cortex-A72 and Cortex-A53.....	8
4.1	Set Key from OTP	10
4.2	Run AES/SM4	11
4.3	Run AES/SM4 by Key on Key Table	12
4.4	Generate MAC	14
4.5	Generate MAC by Key on Key Table	15
4.6	Generate Digest by HASH	16
4.7	Generate Signature by ECDSA.....	17
4.8	Verify Signature by ECDSA.....	18
4.9	Generate Signature by RSASSA_PKCS	19
4.10	Verify Signature by RSASSA_PKCS	20
4.11	Generate Signature by RSASSA_PSS	21
4.12	Verify Signature by RSASSA_PSS	22
4.13	Get Random Number	23
4.14	Write OTP	24
4.15	Get Firmware Version.....	25
5	HSM to Use HSM APIs on Cortex-R5	26
5.1	Set Key from OTP	26
5.2	Run AES/SM4	27
5.3	Run AES/SM4 by Key on Key Table	29
5.4	Generate MAC	31
5.5	Generate MAC by Key on Key Table	33
5.6	Generate Digest by HASH	35
5.7	Generate Signature by ECDSA.....	36
5.8	Verify Signature by ECDSA.....	38
5.9	Generate Signature by RSASSA_PKCS	39
5.10	Verify Signature by RSASSA_PKCS	41
5.11	Generate Signature by RSASSA_PSS	42
5.12	Verify Signature by RSASSA_PSS	43
5.13	Get Random Number	44
5.14	Write OTP	45
5.15	Get Firmware Version.....	46
6	Appendix.....	47
6.1	Request Type	47
6.2	TCC HSM Error Code	47
7	References	48
8	Revision History	49
	Rev. 1.10: 2020-08-05	49
	Rev. 1.00: 2020-07-07	49

Figures

Figure 3.1 Internal Structure of HSM	6
--	---

Tables

Table 1.1 Available SDK	3
Table 1.2 Supported Cryptographic Algorithm	3
Table 4.1 HSM ioctl Command.....	9

1 INTRODUCTION

This document describes a set of C APIs for using HSM (Hardware Security Module) based on TCC805x.

Table 1.1 shows available chipset and SDK version.

Table 1.1 Available SDK

SDK
TCC805x_AndroidP_IVI V1.0.0
TCC805x_Android10_IVI V1.0.0
TCC805x_Linux_IVI V1.0.0

1.1 Motivation

HSM in TCC805x is based on SSS (Samsung Security System) architecture. SSS is an embedded security system targeted for applications requiring strong security. The HSM can be used as a sub-system to enhance the security level of the target device. To support IEE (Isolated Execution Environment), this sub-system provides a dedicated isolated processor, root of trust, secure storage, and cryptographic hardware accelerators.

1.2 Feature

HSM has the following features.

Table 1.2 Supported Cryptographic Algorithm

Category	Function	Bit Mode	Details	Support
Asymmetric Cipher	RSA	1024, 2048, 4096	RSASSA-PKCS1-V1.5 RSASSA-PSS	O
	ECC	256, 384, 512	NIST-Pxxx Brainpool-xxx	O
Symmetric Cipher	AES	128, 192, 256	ECB, CBC, CTR, XTS, CCM, GCM	O
	SM4	128	ECB, CBC	O
HASH	SHA1	160		O
	SHA2	256, 384, 512		O
	SHA3	224, 256, 384, 512		O
	SM3	256		O
MAC	CMAC			O
	GMAC			O
	HMAC			O
	SM3			O

1.3 Abbreviation

■ AES	Advanced Encryption Standard
■ CBC	Cipher-block Chaining
■ CMAC	Cipher-based Message Authentication Code
■ ECB	Electronic Codebook
■ ECDSA	Elliptic Curve Digital Signature Algorithm
■ GMAC	Galois Message Authentication Code
■ HMAC	Hash-based Message Authentication Code
■ HSM	Hardware Security Module
■ IV	Initialization Vector
■ MAC	Message Authentication Code
■ MD5	Message Digest 5
■ MGF	Mask Generation Function
■ NIST	National Institute of Standards and Technology
■ OTP	One-time programmable ROM
■ PSS	Probabilistic Signature Scheme
■ RSA	Rivest-Shamir-Adleman public key cryptosystem
■ RSASSA PKCS	RSA Signature Scheme with Appendix Public Key Cryptography Standards
■ RSASSA PSS	RSA Signature Scheme with Appendix Probabilistic Signature Scheme
■ SHA	Secure Hash Algorithm
■ SM3	Chinese standard hash algorithm
■ SM4	Chinese standard symmetric key block cipher
■ TRNG	True Random Number Generator

2 USE CASES

This chapter describes the applicable use cases when using the HSM.

2.1 Flashing Secure

- Provides protection against flashing of malicious code or data
- Ensures authentic and confidential (IP protection) software updates
- Achieved by: Signature verification of new flashing data/bootloader secured by secure boot process

2.2 Secure Communication

- Provides secure in-vehicle communication that prevents manipulation of communication data between processors
- Achieved by: Uses only message authentication and no encryption/HSM supports symmetric key distribution protocol between the APs to enable secure communication

2.3 Feature Activation

- Ensures that paid features are activated only after a valid authorization/license code is present

3 How to Use HSM

This chapter describes how to use HSM from Cortex-72 and Cortex-53.

3.1 HSM Driver for Linux on Cortex-A72 and Cortex-A53

Cortex-A72 is based on Linux. Therefore, an application should use HSM by Linux kernel driver. The main functions of the HSM provided through the Linux kernel are supported via the ioctl function. The internal structure of HSM is as follows.

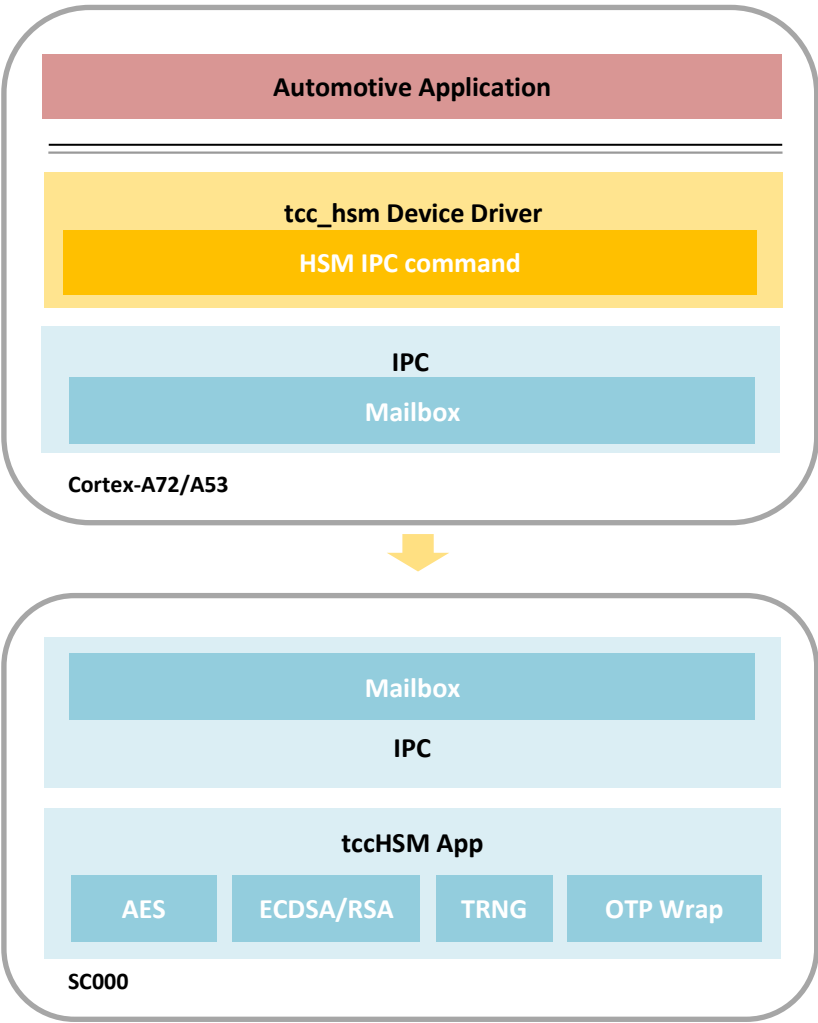


Figure 3.1 Internal Structure of HSM

3.1.1 Name of Driver

At boot time, tcc_hsm driver is generated as tcc_hsm under the /dev folder. You can use the cipher, OTP, and TRNG functions provided by tcc_hsm driver through the usual open, close, and ioctl functions.

```
root@telechips-tcc805x:/dev# ls -l tcc_hsm
crw----- 1 root    root      10, 63 Jul  9 10:39 tcc_hsm
```

4 HOW TO USE IOCTL ON CORTEX-A72 AND CORTEX-A53

The functions provided by the `tcc_hsm` driver are available through the `ioctl` function. The use of the `ioctl` function varies depending on the 2nd argument (`cmd`) and the 3rd argument (`arg`). The command values for distinguishing functions are as follows.

```
enum tcc_hsm_ioctl_cmd
{
    TCCHSM_IOCTL_SET_KEY_FROM_OTP,
    TCCHSM_IOCTL_RUN_AES,
    TCCHSM_IOCTL_RUN_AES_BY_KT,
    TCCHSM_IOCTL_RUN_SM4,
    TCCHSM_IOCTL_RUN_SM4_BY_KT,
    TCCHSM_IOCTL_GEN_CMAC,
    TCCHSM_IOCTL_GEN_CMAC_BY_KT,
    TCCHSM_IOCTL_GEN_GMAC,
    TCCHSM_IOCTL_GEN_GMAC_BY_KT,
    TCCHSM_IOCTL_GEN_HMAC,
    TCCHSM_IOCTL_GEN_HMAC_BY_KT,
    TCCHSM_IOCTL_GEN_SM3_HMAC,
    TCCHSM_IOCTL_GEN_SM3_HMAC_BY_KT,
    TCCHSM_IOCTL_GEN_SHA,
    TCCHSM_IOCTL_GEN_SM3,
    TCCHSM_IOCTL_RUN_ECDSA_SIGN,
    TCCHSM_IOCTL_RUN_ECDSA_VERIFY,
    TCCHSM_IOCTL_RUN_RSASSA_PKCS_SIGN,
    TCCHSM_IOCTL_RUN_RSASSA_PKCS_VERIFY,
    TCCHSM_IOCTL_RUN_RSASSA_PSS_SIGN,
    TCCHSM_IOCTL_RUN_RSASSA_PSS_VERIFY,
    TCCHSM_IOCTL_GET_RNG,
    TCCHSM_IOCTL_WRITE_OTP,
    TCCHSM_IOCTL_GET_VER,
    TCCHSM_IOCTL_MAX
};
```

Table 4.1 describes the functions supported by each command.

Table 4.1 HSM ioctl Command

Command	Function
TCCHSM_IOCTL_SET_KEY_FROM_OTP	Set key written in OTP
TCCHSM_IOCTL_RUN_AES	Execute AES (Advanced Encryption Standard) function
TCCHSM_IOCTL_RUN_AES_BY_KT	Execute AES function with key in key table
TCCHSM_IOCTL_RUN_SM4	Execute SM4 (Chinese standard symmetric key block cipher)
TCCHSM_IOCTL_RUN_SM4_BY_KT	Execute SM4 function with key in key table
TCCHSM_IOCTL_GEN_CMAC	Generate CMAC (Cipher-based Message Authentication Code)
TCCHSM_IOCTL_GEN_CMAC_BY_KT	Generate CMAC with key in key table
TCCHSM_IOCTL_GEN_GMAC	Generate GMAC (Galois Message Authentication Code)
TCCHSM_IOCTL_GEN_GMAC_BY_KT	Generate GMAC with key in key table
TCCHSM_IOCTL_GEN_HMAC	Generate HMAC (Hash-based Message Authentication Code)
TCCHSM_IOCTL_GEN_HMAC_BY_KT	Generate HMAC with key in key table
TCCHSM_IOCTL_GEN_SM3_HMAC	Generate SM3 HMAC (Chinese standard hash algorithm based Message Authentication Code)
TCCHSM_IOCTL_GEN_SM3_HMAC_BY_KT	Generate SM4 HMAC with key in key table
TCCHSM_IOCTL_GEN_SHA	Generate SHA (Secure Hash Algorithm) digest
TCCHSM_IOCTL_GEN_SM3	Generate SM3 (Chinese standard hash algorithm) digest
TCCHSM_IOCTL_RUN_ECDSA_SIGN	Generate signature by ECDSA (Elliptic Curve Digital Signature Algorithm)
TCCHSM_IOCTL_RUN_ECDSA_VERIFY	Verify signature by ECDSA
TCCHSM_IOCTL_RUN_RSASSA_PKCS_SIGN	Generate signature by RSASSA_PKCS (RSA Signature Scheme with Appendix Public Key Cryptography Standards)
TCCHSM_IOCTL_RUN_RSASSA_PKCS_VERIFY	Verify signature by RSASSA_PKCS
TCCHSM_IOCTL_RUN_RSASSA_PSS_SIGN	Generate signature by RSASSA_PSS (RSA Signature Scheme with Appendix Probabilistic Signature Scheme)
TCCHSM_IOCTL_RUN_RSASSA_PSS_VERIFY	Verify signature by RSASSA_PSS
TCCHSM_IOCTL_GET_RNG	Get random number form hardware TRNG (True Random Number Generator)
CCHSM_IOCTL_WRITE_OTP	Write the value in reserved HSM area in OTP
TCCHSM_IOCTL_GET_VER	Get the HSM firmware version

The arguments for each command are set and used as follows.

4.1 Set Key from OTP

Set the key value to use cryptography functions supported by HSM.

Cmd

```
TCCHSM_IOCTL_SET_KEY_FROM_OTP
```

Arg

```
struct tcc_hsm_ioctl_set_key_param {  
    uint32_t addr;  
    uint32_t data_size;  
    uint32_t key_index;  
}
```

addr: [in] Address to load key from OTP
data_size: [in] Size of key
key_index: [in] Index of keyslot. The number of keyslots is 16.

Example

```
struct tcc_hsm_ioctl_set_key_param param;  
  
param.addr = 0x00000000;  
param.data_size = 16;  
param.key_index = 1;  
  
ioctl(fd, TCCHSM_IOCTL_SET_KEY_FROM_OTP, &param);
```

4.2 Run AES/SM4

Run AES/SM4 encryption/decryption by cryptography functions supported by HSM.

Cmd

```
TCCISM_IOCTL_RUN_AES
TCCISM_IOCTL_RUN_SM4
```

Arg

```
enum tcc_hsm_ioctl_obj_id_aes {
    OID_AES_ENCRYPT = 0x00000000,
    OID_AES_DECRYPT = 0x01000000,
    OID_AES_ECB_128 = 0x00100008,
    OID_AES_ECB_192 = 0x00180008,
    OID_AES_ECB_256 = 0x00200008,
    OID_AES_CBC_128 = 0x00100108,
    OID_AES_CBC_192 = 0x00180108,
    OID_AES_CBC_256 = 0x00200108,
    OID_AES_CTR_128 = 0x00100208,
    OID_AES_CTR_192 = 0x00180208,
    OID_AES_CTR_256 = 0x00200208,
    OID_AES_XTS_128 = 0x00100308,
    OID_AES_XTS_256 = 0x00200308,
    OID_AES_CCM_128 = 0x00101008,
    OID_AES_CCM_192 = 0x00181008,
    OID_AES_CCM_256 = 0x00201008,
    OID_AES_GCM_128 = 0x00101108,
    OID_AES_GCM_192 = 0x00181108,
    OID_AES_GCM_256 = 0x00201108,
};

enum tcc_hsm_ioctl_obj_id_sm4 {
    OID_SM4_ENCRYPT = 0x00000000,
    OID_SM4_DECRYPT = 0x01000000,
    OID_SM4_ECB_128 = 0x00100008,
    OID_SM4_CBC_128 = 0x00100108,
};

struct tcc_hsm_ioctl_aes_param {
    uint32_t obj_id;
    uint8_t key[TCC_HSM_AES_KEY_SIZE];
    uint32_t key_size;
    uint8_t iv[TCC_HSM_AES_IV_SIZE];
    uint32_t iv_size;
    uint64_t src;
    uint32_t src_size;
    uint64_t dst;
    uint32_t dst_size;
    uint32_t dma;
}
```

obj_id:	[in] Object ID to distinguish symmetric functions.
key:	[in] Buffer of AES/SM4 key
key_size:	[in] Size of AES/SM4 key
iv:	[in] Buffer of initialization vector
iv_size:	[in] Size of initialization vector
src:	[in] Buffer pointer of input data
src_size:	[in] Size of src
dst:	[out] Buffer pointer of output data
dst_size:	[in] Size of dst
dma:	[in] 0: src and dst are virtual memory address, others: src and dst are physical memory address

Example

```

struct tcc_hsm_ioctl_aes_param param;

param.obj_id = OID_AES_ENCRYPT | OID_AES_CBC_128;
memcpy(param.key, key, 16);
param.key_size = 16;
memcpy(param.iv, iv, 16);
param.iv_size = iv;
param.src = src;
param.src_size = sizeof(src);
param.dst = dst;
param.dst_size = sizeof(dst);
param.dma = 1;

ioctl(fd, TCCHSM_IOCTL_RUN_AES, &param);

```

4.3 Run AES/SM4 by Key on Key Table

Run AES/SM4 encryption/decryption by key loaded on key table.

Cmd

```

TCCHSM_IOCTL_RUN_AES_BY_KT
TCCHSM_IOCTL_RUN_SM4_BY_KT

```

Arg

```

enum tcc_hsm_ioctl_obj_id_aes {
    OID_AES_ENCRYPT = 0x00000000,
    OID_AES_DECRYPT = 0x01000000,
    OID_AES_ECB_128 = 0x00100008,
    OID_AES_ECB_192 = 0x00180008,
    OID_AES_ECB_256 = 0x00200008,
    OID_AES_CBC_128 = 0x00100108,
    OID_AES_CBC_192 = 0x00180108,
    OID_AES_CBC_256 = 0x00200108,
    OID_AES_CTR_128 = 0x00100208,
    OID_AES_CTR_192 = 0x00180208,
    OID_AES_CTR_256 = 0x00200208,
    OID_AES_XTS_128 = 0x00100308,
    OID_AES_XTS_256 = 0x00200308,
    OID_AES_CCM_128 = 0x00101008,
    OID_AES_CCM_192 = 0x00181008,
    OID_AES_CCM_256 = 0x00201008,
    OID_AES_GCM_128 = 0x00101108,
    OID_AES_GCM_192 = 0x00181108,
    OID_AES_GCM_256 = 0x00201108,
};

enum tcc_hsm_ioctl_obj_id_sm4 {
    OID_SM4_ENCRYPT = 0x00000000,
    OID_SM4_DECRYPT = 0x01000000,
    OID_SM4_ECB_128 = 0x00100008,
    OID_SM4_CBC_128 = 0x00100108,
};

struct tcc_hsm_ioctl_aes_by_kt_param {
    uint32_t obj_id;
    uint32_t key_index;
    uint8_t iv[TCC_HSM_AES_IV_SIZE];
};

```

```

uint32_t iv_size;
uint64_t src;
uint32_t src_size;
uint64_t dst;
uint32_t dst_size;
uint32_t dma;
}

```

obj_id: [in] Object ID to distinguish symmetric functions.
 key_index: [in] Index of AES/SM4 key on key table
 iv: [in] Buffer of initialization vector
 iv_size: [in] Size of initialization vector
 src: [in] Buffer pointer of input data
 src_size: [in] Size of src
 dst: [out] Buffer pointer of output data
 dst_size: [in] Size of dst
 dma: [in] 0: src and dst are virtual memory address, others: src and dst are physical memory address

Example

```

struct tcc_hsm_ioctl_aes_by_kt_param param;

param.obj_id = OID_AES_ENCRYPT | OID_AES_CBC_128;
param.key_index = 0;
memcpy(param.iv, iv, 16);
param.iv_size = iv;
param.src = src;
param.src_size = sizeof(src);
param.dst = dst;
param.dst_size = sizeof(dst);
param.dma = 1;

ioctl(fd, TCCHSM_IOCTL_RUN_AES_BY_KT, &param);

```

4.4 Generate MAC

Generate MAC (Message Authentication Code) by CMAC/GMAC/HMAC/SM3_HMAC.

Cmd

```
TCCHSM_IOCTL_GEN_CMAC
TCCHSM_IOCTL_GEN_GMAC
TCCHSM_IOCTL_GEN_HMAC
TCCHSM_IOCTL_GEN_SM3_HMAC
```

Arg

```
enum tcc_hsm_ioctl_obj_id_hmac {
    OID_HMAC_SHA1_160 = 0x00011100,
    OID_HMAC_SHA2_224 = 0x00012200,
    OID_HMAC_SHA2_256 = 0x00012300,
    OID_HMAC_SHA2_384 = 0x00012400,
    OID_HMAC_SHA2_512 = 0x00012500,
    OID_HMAC_SHA3_224 = 0x00013200,
    OID_HMAC_SHA3_256 = 0x00013300,
    OID_HMAC_SHA3_384 = 0x00013400,
    OID_HMAC_SHA3_512 = 0x00013500,
};

struct tcc_hsm_ioctl_mac_param {
    uint32_t obj_id;
    uint8_t key[TCC_HSM_MAC_KEY_SIZE];
    uint32_t key_size;
    uint64_t src;
    uint32_t src_size;
    uint8_t mac[TCC_HSM_MAC_MSG_SIZE];
    uint32_t mac_size;
    uint32_t dma;
};
```

obj_id:	[in] Object ID to distinguish HMAC functions. Otherwise do not use this field.
key:	[in] Buffer of CMAC/GMAC/HMAC/SM3_HMAC key
key_size:	[in] Size of CMAC/GMAC/HMAC/SM3_HMAC key
src:	[in] Buffer pointer of input data
src_size:	[in] Size of src
mac:	[out] Buffer pointer of mac
mac_size:	[in] Size of mac
dma:	[in] 0: src is virtual memory address, others: src is physical memory address

Example

```
struct tcc_hsm_ioctl_mac_param param;

param.obj_id = OID_HMAC_SHA2_256;
memcpy(param.key, key, 32);
param.key_size = 32;
param.src = src;
param.src_size = sizeof(src);
param.dma = 1;

ioctl(fd, TCCHSM_IOCTL_GEN_HMAC, &param);
```

4.5 Generate MAC by Key on Key Table

Generate MAC by key loaded on key table.

Cmd

```
TCCHSM_IOCTL_GEN_CMAC_BY_KT
TCCHSM_IOCTL_GEN_GMAC_BY_KT
TCCHSM_IOCTL_GEN_HMAC_BY_KT
TCCHSM_IOCTL_GEN_SM3_HMAC_BY_KT
```

Arg

```
enum tcc_hsm_ioctl_obj_id_hmac {
    OID_HMAC_SHA1_160 = 0x00011100,
    OID_HMAC_SHA2_224 = 0x00012200,
    OID_HMAC_SHA2_256 = 0x00012300,
    OID_HMAC_SHA2_384 = 0x00012400,
    OID_HMAC_SHA2_512 = 0x00012500,
    OID_HMAC_SHA3_224 = 0x00013200,
    OID_HMAC_SHA3_256 = 0x00013300,
    OID_HMAC_SHA3_384 = 0x00013400,
    OID_HMAC_SHA3_512 = 0x00013500,
};

struct tcc_hsm_ioctl_mac_by_kt_param {
    uint32_t obj_id;
    uint32_t key_index;
    uint64_t src;
    uint32_t src_size;
    uint8_t mac[TCC_HSM_MAC_MSG_SIZE];
    uint32_t mac_size;
    uint32_t dma;
};
```

obj_id:	[in] Object ID to distinguish HMAC functions. Otherwise do not use this field.
key_index:	[in] Index of CMAC/GMAC/HMAC/SM3_HMAC key
src:	[in] Buffer pointer of input data
src_size:	[in] Size of src
mac:	[out] Buffer pointer of mac
mac_size:	[in] Size of mac
dma:	[in] 0: src is virtual memory address, others: src is physical memory address

Example

```
struct tcc_hsm_ioctl_mac_param param;

param.obj_id = OID_HMAC_SHA2_256;
param.key_index = 1;
param.src = src;
param.src_size = sizeof(src);
param.dma = 1;

ioctl(fd, TCCHSM_IOCTL_GEN_HMAC, &param);
```

4.6 Generate Digest by HASH

Generate digest by SHA/SM3.

Cmd

```
TCCHSM_IOCTL_GEN_SHA
TCCHSM_IOCTL_GEN_SM3
```

Arg

```
enum tcc_hsm_ioctl_obj_id_hash {
    OID_SHA1_160 = 0x00001100,
    OID_SHA2_224 = 0x00002200,
    OID_SHA2_256 = 0x00002300,
    OID_SHA2_384 = 0x00002400,
    OID_SHA2_512 = 0x00002500,
    OID_SHA3_224 = 0x00003200,
    OID_SHA3_256 = 0x00003300,
    OID_SHA3_384 = 0x00003400,
    OID_SHA3_512 = 0x00003500,
    OID_SM3_256 = 0x01002300,
};

struct tcc_hsm_ioctl_hash_param {
    uint32_t obj_id;
    uint64_t src;
    uint32_t src_size;
    uint8_t digest[TCC_HSM_HASH_DIGEST_SIZE];
    uint32_t digest_size;
    uint32_t dma;
};
```

obj_id: [in] Object ID to distinguish SHA/SM3 functions
 src: [in] Buffer pointer of input data
 src_size: [in] Size of src
 digest: [out] Buffer pointer of result digest
 digest_size: [in] Size of digest
 dma: [in] 0: src is virtual memory address, others: src is physical memory address

Example

```
struct tcc_hsm_ioctl_hash_param param;

param.obj_id = OID_SHA2_256;;
param.src = src;
param.src_size = sizeof(src);
param.dma = 1;

ioctl(fd, TCCHSM_IOCTL_GEN_SHA, &param);
```

4.7 Generate Signature by ECDSA

Generate signature by ECDSA.

Cmd

```
TCCHSM_IOCTL_RUN_ECDSA_SIGN
```

Arg

```
enum tcc_hsm_ioctl_obj_id_ecc {
    OID_ECC_P256 = 0x00000013,
    OID_ECC_P384 = 0x00000014,
    OID_ECC_P256 = 0x00000013,
    OID_ECC_P384 = 0x00000014,
    OID_ECC_P521 = 0x00000015,
    OID_ECC_BP256 = 0x00000053,
    OID_ECC_BP384 = 0x00000054,
    OID_ECC_BP512 = 0x00000055,
    OID_SM2_256_SM3_256 = 0x010023A3,
};

struct tcc_hsm_ioctl_ecdsa_param {
    uint32_t obj_id;
    uint8_t key[TCC_HSM_ECDSA_KEY_SIZE];
    uint32_t key_size;
    uint8_t digest[TCC_HSM_ECDSA_DIGEST_SIZE];
    uint32_t digest_size;
    uint8_t sig[TCC_HSM_ECDSA_SIGN_SIZE];
    uint32_t sig_size;
};
```

obj_id:	[in] Object ID to distinguish ECDSA algorithm.
key:	[in] Private key
key_size:	[in] Size of private key
digest:	[in] Buffer pointer of digest
dig_size:	[in] Size of digest
sig:	[out] Buffer pointer of ECDSA signature
sig_size:	[in] Size of signature

Example

```
struct tcc_hsm_ioctl_ecdsa_param param;

param.obj_id = OID_ECC_BP256;
memcpy(param.key, privatekey, 32);
param.key_size = 32;
memcpy(param.digest, digest, 32);
param.digest_size = 32;

ioctl(fd, TCCHSM_IOCTL_RUN_ECDSA_SIGN, &param);
```

4.8 Verify Signature by ECDSA

Verify signature by ECDSA.

Cmd

```
TCC_HSM_IOCTL_RUN_ECDSA_VERIFY
```

Arg

```
enum tcc_hsm_ioctl_obj_id_ecc {
    OID_ECC_P256 = 0x00000013,
    OID_ECC_P384 = 0x00000014,
    OID_ECC_P256 = 0x00000013,
    OID_ECC_P384 = 0x00000014,
    OID_ECC_P521 = 0x00000015,
    OID_ECC_BP256 = 0x00000053,
    OID_ECC_BP384 = 0x00000054,
    OID_ECC_BP512 = 0x00000055,
    OID_SM2_256_SM3_256 = 0x010023A3,
};

struct tcc_hsm_ioctl_ecdsa_param {
    uint32_t obj_id;
    uint8_t key[TCC_HSM_ECDSA_KEY_SIZE];
    uint32_t key_size;
    uint8_t digest[TCC_HSM_ECDSA_DIGEST_SIZE];
    uint32_t digest_size;
    uint8_t sig[TCC_HSM_ECDSA_SIGN_SIZE];
    uint32_t sig_size;
};
```

obj_id: [in] Object ID to distinguish ECDSA algorithm.
 key: [in] Public key
 key_size: [in] Size of public key
 digest: [in] Buffer pointer of digest
 digest_size: [in] Size of digest
 sig: [in] Buffer pointer of ECDSA signature
 sig_size: [in] Size of signature

Example

```
struct tcc_hsm_ioctl_ecdsa_param param;

param.obj_id = OID_ECC_BP256;
memcpy(param.key, publickey, 64);
param.key_size = 64;
memcpy(param.digest, digest, 32);
param.digest_size = 32;
memcpy(param.sig, signature, 64);
param.sig_size = 64;

ioctl(fd, TCC_HSM_IOCTL_RUN_ECDSA_VERIFY, &param);
```

4.9 Generate Signature by RSASSA_PKCS

Generate signature by RSASSA_PKCS.

Cmd

```
TCCISM_IOCTL_RUN_RSASSA_PKCS_SIGN
```

Arg

```
struct tcc_hsm_ioctl_rsassa_param {
    uint32_t obj_id
    uint8_t modN[TCC_HSM_RSA_MODN_SIZE];
    uint32_t modN_size;
    uint64_t key;
    uint32_t key_size;
    uint8_t digest[TCC_HSM_RSA_DIGEST_SIZE];
    uint32_t digest_size;
    uint8_t sig[TCC_HSM_RSA_SIGN_SIZE];
    uint32_t sig_size;
};
```

obj_id:	[in] Object ID to distinguish RSA algorithm.
modN:	[in] Modulus N for RSA
modN_size:	[in] Size of modN
key:	[in] Private key
key_size:	[in] Size of private key
digest:	[in] Buffer pointer of digest
digest_size:	[in] Size of digest
sig:	[out] Buffer pointer of RSA signature
sig_size:	[in] Size of signature

Example

```
struct tcc_hsm_ioctl_rsa_param param;

param.obj_id = 0;
memcpy(param.modN, modN, 128);
param.modN_size = 128;
param.key = privatekey;
param.key_size = 128;
memcpy(param.digest, digest, 32);
param.digest_size = 32;

ioctl(fd, TCCISM_IOCTL_RUN_RSASSA_PKCS_SIGN, &param);
```

4.10 Verify Signature by RSASSA_PKCS

Verify signature by RSASSA_PKCS.

Cmd

```
TCCHSM_IOCTL_RUN_RSASSA_PKCS_VERIFY
```

Arg

```
struct tcc_hsm_ioctl_rsassa_param {
    uint8_t modN[TCC_HSM_RSA_MODN_SIZE];
    uint32_t modN_size;
    uint64_t key;
    uint32_t key_size;
    uint8_t digest[TCC_HSM_RSA_DIGEST_SIZE];
    uint32_t digest_size;
    uint8_t sig[TCC_HSM_RSA_SIGN_SIZE];
    uint32_t sig_size;
};
```

obj_id:	[in] Object ID to distinguish RSA algorithm.
modN:	[in] Modulus N for RSA
modN_size:	[in] Size of modN
key:	[in] Public key
key_size:	[in] Size of public key
digest:	[in] Buffer pointer of digest
digest_size:	[in] Size of digest
sign:	[in] Buffer pointer of RSA signature
sig_size:	[in] Size of signature

Example

```
struct tcc_hsm_ioctl_rsa_param param;

param.obj_id = 0;
memcpy(param.modN, modN, 128);
param.modN_size = 128;
param.key = privatekey;
param.key_size = 128;
memcpy(param.digest, digest, 32);
param.digest_size = 32;
memcpy(param.sig, sign, 128);
param.sig_size = 128;

ioctl(fd, TCCHSM_IOCTL_RUN_RSASSA_PKCS_VERIFY, &param);
```

4.11 Generate Signature by RSASSA_PSS

Generate signature by RSASSA_PSS.

Cmd

```
TCCHSM_IOCTL_RUN_RSASSA_PSS_SIGN
```

Arg

```
struct tcc_hsm_ioctl_rsa_param {
    uint8_t modN[TCC_HSM_RSA_MODN_SIZE];
    uint32_t modN_size;
    uint64_t key;
    uint32_t key_size;
    uint8_t digest[TCC_HSM_RSA_DIGEST_SIZE];
    uint32_t digest_size;
    uint8_t sig[TCC_HSM_RSA_SIGN_SIZE];
    uint32_t sig_size;
};
```

obj_id:	[in] Object ID to distinguish RSA algorithm.
modN:	[in] Modulus N for RSA
modN_size:	[in] Size of modN
key:	[in] Private key
key_size:	[in] Size of private key
digest:	[in] Buffer pointer of digest
digest_size:	[in] Size of digest
sig:	[out] Buffer pointer of RSA signature
sig_size:	[in] Size of signature

Example

```
struct tcc_hsm_ioctl_rsa_param param;

param.obj_id = RSASSA_PSS_OID_HASH;
memcpy(param.modN, modN, 128);
param.modN_size = 128;
param.key = privatekey;
param.key_size = 128;
memcpy(param.digest, digest, 32);
param.digest_size = 32;

ioctl(fd, TCCHSM_IOCTL_RUN_RSASSA_PSS_SIGN, &param);
```

4.12 Verify Signature by RSASSA_PSS

Verify signature by RSASSA_PSS.

Cmd

```
TCCHSM_IOCTL_RUN_RSASSA_PSS_VERIFY
```

Arg

```
struct tcc_hsm_ioctl_rsa_param {
    uint8_t modN[TCC_HSM_RSA_MODN_SIZE];
    uint32_t modN_size;
    uint64_t key;
    uint32_t key_size;
    uint8_t digest[TCC_HSM_RSA_DIGEST_SIZE];
    uint32_t digest_size;
    uint8_t sig[TCC_HSM_RSA_SIGN_SIZE];
    uint32_t sig_size;
};
```

obj_id:	[in] Object ID to distinguish RSA algorithm.
modN:	[in] Modulus N for RSA
modN_size:	[in] Size of modN
key:	[in] Public key
key_size:	[in] Size of public key
digest:	[in] Buffer pointer of digest
digest_size:	[in] Size of digest
sign:	[out] Buffer pointer of RSA signature
sig_size:	[in] Size of signature

Example

```
struct tcc_hsm_ioctl_rsa_param param;

param.obj_id = RSASSA_PSS_OID_HASH;
memcpy(param.modN, modN, 128);
param.modN_size = 128;
param.key = privatekey;
param.key_size = 128;
memcpy(param.digest, digest, 32);
param.digest_size = 32;
memcpy(param.sig, sig, 128);
param.sig_size = 128;

ioctl(fd, TCCHSM_IOCTL_RUN_RSASSA_PSS_VERIFY, &param);
```

4.13 Get Random Number

Get random number through hardware TRNG block in HSM.

Cmd

```
TCCHSM_IOCTL_GET_RNG
```

Arg

```
struct tcc_hsm_ioctl_rng_param {  
    uint64_t rng;  
    uint32_t rng_size;  
};
```

rng: [out] Buffer pointer to get random number
rng_size: [in] Size of random number. Maximum size is 32 bytes.

Example

```
struct tcc_hsm_ioctl_rng_param param;  
  
uint32_t buf[2] = {0, };  
  
param.rng = buf;  
param.size = sizeof(uint32_t) * 2;  
  
ioctl(fd, TCCHSM_IOCTL_GEN_RNG, &param);
```

4.14 Write OTP

Write the value in reserved HSM area in OTP.

Cmd

```
TCCHSM_IOCTL_WRITE_OTP
```

Arg

```
struct tcc_hsm_ioctl_write_param {  
    uint32_t oddr;  
    uint64_t data;  
    uint32_t data_size;  
};
```

addr: [in] OTP address to be programmed. Address starts from 0 to 0x1A00, and the size is 1024 bytes.
data: [in] Buffer pointer of data to program OTP
data_size: [in] Size of data

Example

```
struct tcc_hsm_ioctl_otp_param param;  
  
uint32_t buf[2] = {0x12345678, 0x9ABCDEF0};  
  
param.addr = 0x0;  
param.data = buf;  
param.data_size = 16;  
  
ioctl(fd, TCCHSM_IOCTL_WRITE_OTP, &param);
```

4.15 Get Firmware Version

Get the HSM firmware version.

Cmd

```
TCCHSM_IOCTL_GET_VER
```

Arg

```
struct tcc_hsm_ioctl_version_param {  
    uint32_t x;  
    uint32_t y;  
    uint32_t z;  
};
```

Example

```
struct tcc_hsm_ioctl_version_param param;  
  
param.x = 0;  
param.y = 0;  
param.z = 0;  
  
ioctl(fd, TCCHSM_IOCTL_GET_VERSION, &param);
```

5 HSM TO USE HSM APIs ON CORTEX-R5

Cortex-R5 is based on FreeRTOS to use HSM in your application.

The functions provided by the MICOM firmware are available through the HSM manager service. The functions are as follows.

5.1 Set Key from OTP

Set the key value to use cryptography functions supported by HSM.

Function

```
int32 hsm_set_key(uint32 req, uint32 addr, uint32 data_size, uint32 key_index)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- addr: [in] Address to load key from OTP
- data_size: [in] Size of key
- key_index: [in] Index of keyslot. The number of keyslots is 16.

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_set_key_test(uint8 cmd)
{
    int32 ret = TCCHSM_ERR;
    uint32 req = 0, key_index = 0, addr = 0, data_size = 0;

    if (cmd == TCCHSM_SET_KEY_FROM_OTP) {
        addr = 0x00000030;
        req = REQ_HSM_SET_KEY_FROM_OTP;
    } else if (cmd == TCCHSM_SET_KEY_FROM_SNOR) {
        addr = 0x00000000;
        req = REQ_HSM_SET_KEY_FROM_SNOR;
    } else {
        ELOG("invalid cmd(%d)\n", cmd);
        return ret;
    }

    data_size = sizeof(key);
    key_index = 0;

    ret = hsm_set_key(req, addr, data_size, key_index);
    if (ret) {
        ELOG("hsm_set_key_test Err(%d)\n", ret);
        return ret;
    }

    if (cmd == TCCHSM_SET_KEY_FROM_OTP) {
        addr = 0x00000040;
    } else if (cmd == TCCHSM_SET_KEY_FROM_SNOR) {
        addr = 0x00000100;
    }
}
```

```

    data_size = sizeof(mac_key);
    key_index = 1;

    ret = hsm_set_key(req, addr, data_size, key_index);
    if (ret) {
        ELOG("hsm_set_key test fail(%d)\n", ret);
        return ret;
    }

    DLOG("hsm_set_key(addr=0x%x) Success\n", addr);
    return ret;
}

```

5.2 Run AES/SM4

Run AES/SM4 encryption/decryption by cryptography functions supported by HSM.

Function

```

int32 hsm_run_aes(
    uint32 req, uint32 obj_id, uint8 *key, uint32 key_size, uint8 *iv, uint32 iv_size,
    uint8 *src_addr, uint32 src_size, uint8 *dst_addr, uint32 dst_size)

```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- obj_id: [in] Object ID to distinguish symmetric functions
- key: [in] Buffer of AES/SM4 key
- key_size: [in] Size of AES/SM4 key
- iv: [in] Buffer of initialization vector
- iv_size: [in] Size of initialization vector
- src: [in] Buffer pointer of input data
- src_size: [in] Size of src
- dst: [out] Buffer pointer of output data
- dst_size: [in] Size of dst

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```

static int32 hsm_run_aes_test(uint8 cmd)
{
    uint32_t obj_id = 0, key_size = 0, src_size = 0, dst_size = 0;
    uint32_t req = 0;
    uint8_t dst[32] = {0};
    int32 ret = TCCHSM_ERR;

    switch (cmd) {
    case TCCHSM_RUN_AES:
        /* AES CBC */
        req = REQ_HSM_RUN_AES;
        obj_id = HSM_AES_ECB_128 | HSM_AES_ENCRYPT;
        dst_size = sizeof(aes_ECB_cipher);
        src_size = sizeof(plain_data);
        key_size = sizeof(key);
    }
}

```

```

dst, dst_size);
    ret = hsm_run_aes(req, obj_id, key, key_size, NULL, 0, plain_data, src_size,
dst, dst_size);
    if (ret) {
        ELOG("hsm_run_aes test fail(%d)\n", ret);
        return TCCHSM_ERR;
    }
    if (memcmp(aes_ECB_cipher, dst, dst_size) != 0) {
        HexDump(aes_ECB_cipher, dst_size);
        HexDump(dst, dst_size);
        ELOG("Wrong cipher data\n");
        return TCCHSM_ERR;
    }
    DLOG("RUN_AES Success\n");
    break;

case TCCHSM_RUN_SM4:
    /* SM4 ECB */
    req = REQ_HSM_RUN_SM4;
    obj_id = (OID_SM4_ECB_128 | OID_SM4_ENCRYPT);
    dst_size = sizeof(sm4_ECB_cipher);
    src_size = sizeof(plain_data);
    key_size = sizeof(key);
    ret = hsm_run_aes(req, obj_id, key, key_size, NULL, 0, plain_data, src_size,
dst, dst_size);
    if (ret) {
        ELOG("hsm_run_aes test fail(%d)\n", ret);
        return ret;
    }
    if (memcmp(sm4_ECB_cipher, dst, dst_size) != 0) {
        HexDump(sm4_ECB_cipher, dst_size);
        HexDump(dst, dst_size);
        ELOG("Wrong cipher data\n");
        return TCCHSM_ERR;
    }
    DLOG("RUN_SM4 Success\n");
    break;

default:
    ELOG("Wrong cmd type\n");
    break;
}

return ret;
}

```

5.3 Run AES/SM4 by Key on Key Table

Run AES/SM4 encryption/decryption by key loaded on key table.

Function

```
int32 hsm_run_aes_by_kt(
    uint32 req, uint32 obj_id, uint32 key_index, uint8 *iv, uint32 iv_size, uint8
    *src_addr, uint32 src_size, uint8 *dst_addr, uint32 dst_size)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- obj_id: [in] Object ID to distinguish symmetric functions
- key_index: [in] Index of AES/SM4 key on key table
- iv: [in] Buffer of initialization vector
- iv_size: [in] Size of initialization vector
- src: [in] Buffer pointer of input data
- src_size: [in] Size of src
- dst: [out] Buffer pointer of output data
- dst_size: [in] Size of dst

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_run_aes_by_kt_test(uint8 cmd)
{
    uint32_t obj_id = 0, src_size = 0, dst_size = 0;
    uint32_t req = 0, key_index = 0;
    uint8_t dst[32] = {0};
    int32 ret = TCCHSM_ERR;

    switch (cmd) {
    case TCCHSM_RUN_AES_BY_KT:
        /* AES CBC */
        req = REQ_HSM_RUN_AES_BY_KT;
        obj_id = HSM_AES_ECB_128 | HSM_AES_ENCRYPT;
        dst_size = sizeof(aes_ECB_cipher);
        src_size = sizeof(plain_data);
        key_index = 0;

        ret =
            hsm_run_aes_by_kt(req, obj_id, key_index, NULL, 0, plain_data,
src_size, dst, dst_size);
        if (ret) {
            ELOG("hsm_run_aes test fail(%d)\n", ret);
            return TCCHSM_ERR;
        }
        if (memcmp(aes_ECB_cipher, dst, dst_size) != 0) {
            HexDump(aes_ECB_cipher, dst_size);
            HexDump(dst, dst_size);
            ELOG("Wrong cipher data\n");
            return TCCHSM_ERR;
        }
        DLOG("RUN_AES_BY_KT Success\n");
        break;
    }
```

```
case TCCHSM_RUN_SM4_BY_KT:
    /* SM4 ECB */
    req = REQ_HSM_RUN_SM4_BY_KT;
    obj_id = (OID_SM4_ECB_128 | OID_SM4_ENCRYPT);
    dst_size = sizeof(sm4_ECB_cipher);
    src_size = sizeof(plain_data);
    key_index = 0;

    ret =
src_size, dst, dst_size);
    if (ret) {
        ELOG("hsm_run_aes test fail\n");
        return TCCHSM_ERR;
    }
    if (memcmp(sm4_ECB_cipher, dst, dst_size) != 0) {
        HexDump(sm4_ECB_cipher, dst_size);
        HexDump(dst, dst_size);
        ELOG("Wrong cipher data\n");
        return TCCHSM_ERR;
    }
    DLOG("RUN_SM4_BY_KT Success\n");
    break;

default:
    ELOG("Wrong cmd type\n");
    break;
}

return ret;
}
```

5.4 Generate MAC

Generate MAC by CMAC/GMAC/HMAC/SM3_HMAC.

Function

```
int32 hsm_gen_mac(
    uint32 req, uint32 obj_id, uint8 *key, uint32 key_size, uint8 *src_addr, uint32
src_size, uint8 *mac_addr, uint32 mac_size)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- obj_id: [in] Object ID to distinguish HMAC functions.
- key: [in] Buffer of CMAC/GMAC/HMAC/SM3_HMAC key
- key_size: [in] Size of CMAC/GMAC/HMAC/SM3_HMAC key
- src: [in] Buffer pointer of input data
- src_size: [in] Size of src
- mac: [out] Buffer pointer of mac
- mac_size: [in] Size of mac

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_gen_mac_test(uint8 cmd)
{
    uint32_t obj_id = 0, key_size = 0, src_size = 0, mac_size = 0;
    uint32_t req = 0;
    uint8_t mac[32] = {0};
    int32 ret = TCCHSM_ERR;

    switch (cmd) {
    case TCCHSM_GEN_CMAC:
        req = REQ_HSM_GEN_CMAC;
        obj_id = 0;
        src_size = sizeof(plain_data);
        key_size = sizeof(key);
        mac_size = sizeof(cmac_out);
        ret = hsm_gen_mac(req, obj_id, key, key_size, plain_data, src_size, mac,
mac_size);
        if (ret != TCCHSM_OK) {
            ELOG("hsm_gen_mac test fail\n");
            return TCCHSM_ERR;
        }
        if (memcmp(cmac_out, mac, mac_size) != 0) {
            HexDump(cmac_out, mac_size);
            HexDump(mac, mac_size);
            ELOG("Wrong cipher data\n");
            return TCCHSM_ERR;
        }
        DLOG("GEN_CMAC Success\n");
        break;

    case TCCHSM_GEN_GMAC:
        ELOG("Not Yet\n");
        break;
    }
```

```

        case TCCHSM_GEN_HMAC:
            req = REQ_HSM_GEN_HMAC;
            obj_id = OID_HMAC_SHA1_160;
            src_size = sizeof(plain_data);
            key_size = sizeof(mac_key);
            mac_size = 20;
            ret = hsm_gen_mac(req, obj_id, mac_key, key_size, plain_data, src_size, mac,
mac_size);
            if (ret) {
                ELOG("hsm_gen_hmac test fail\n");
                return TCCHSM_ERR;
            }
            if (memcmp(hmac_out, mac, mac_size) != 0) {
                HexDump(hmac_out, mac_size);
                HexDump(mac, mac_size);
                ELOG("Wrong cipher data\n");
                return TCCHSM_ERR;
            }
            DLOG("TCCHSM_GEN_HMAC Success\n");
            break;

        case TCCHSM_GEN_SM3_HMAC:
            req = REQ_HSM_GEN_SM3_HMAC;
            obj_id = 0;
            src_size = sizeof(plain_data);
            key_size = sizeof(mac_key);
            mac_size = 32;
            ret = hsm_gen_mac(req, obj_id, mac_key, key_size, plain_data, src_size, mac,
mac_size);
            if (ret) {
                ELOG("hsm_gen_sm3_hmac test fail\n");
                return TCCHSM_ERR;
            }
            if (memcmp(sm3_hmac_out, mac, mac_size) != 0) {
                HexDump(sm3_hmac_out, mac_size);
                HexDump(mac, mac_size);
                ELOG("Wrong cipher data\n");
                return TCCHSM_ERR;
            }
            DLOG("TCCHSM_GEN_SM3_HMAC Success\n");
            break;

        default:
            ELOG("Wrong cmd type\n");
            break;
    }

    return ret;
}

```

5.5 Generate MAC by Key on Key Table

Generate MAC by key loaded on key table.

Function

```
int32 hsm_gen_mac_by_kt(
    uint32 req, uint32 obj_id, uint32 key_index, uint8 *src_addr, uint32 src_size, uint8
    *mac_addr, uint32 mac_size)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- obj_id: [in] Object ID to distinguish HMAC functions.
- key_index: [in] Index of CMAC/GMAC/HMAC/SM3_HMAC key
- src: [in] Buffer pointer of input data
- src_size: [in] Size of src
- mac: [out] Buffer pointer of mac
- mac_size: [in] Size of mac

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_gen_mac_by_kt_test(uint8 cmd)
{
    uint32_t obj_id = 0, key_index = 0, src_size = 0, mac_size = 0;
    uint32_t req = 0;
    uint8_t mac[32] = {0};
    int32 ret = TCCHSM_ERR;

    switch (cmd) {
        case TCCHSM_GEN_CMV_BY_KT:
            req = REQ_HSM_GEN_CMV_BY_KT;
            obj_id = 0;
            src_size = sizeof(plain_data);
            key_index = 0;
            mac_size = sizeof(cmac_out);
            ret = hsm_gen_mac_by_kt(req, obj_id, key_index, plain_data, src_size, mac,
mac_size);
            if (ret) {
                ELOG("hsm_gen_cmac test fail\n");
                return TCCHSM_ERR;
            }
            if (memcmp(cmac_out, mac, mac_size) != 0) {
                HexDump(cmac_out, mac_size);
                HexDump(mac, mac_size);
                ELOG("Wrong cipher data\n");
                return TCCHSM_ERR;
            }
            DLOG("GEN_CMV_BY_KT Success\n");
            break;

        case TCCHSM_GEN_GMAC_BY_KT:
            ELOG("Not Yet\n");
            break;

        case TCCHSM_GEN_HMAC_BY_KT:
```

```

        req = REQ_HSM_GEN_HMAC_BY_KT;
        obj_id = OID_HMAC_SHA1_160;
        src_size = sizeof(plain_data);
        key_index = 1;
        mac_size = 20;
        ret = hsm_gen_mac_by_kt(req, obj_id, key_index, plain_data, src_size, mac,
mac_size);

        if (ret) {
            ELOG("hsm_gen_hmac test fail\n");
            return TCCHSM_ERR;
        }
        if (memcmp(hmac_out, mac, mac_size) != 0) {
            HexDump(hmac_out, mac_size);
            HexDump(mac, mac_size);
            ELOG("Wrong cipher data\n");
            return TCCHSM_ERR;
        }
        DLOG("TCCHSM_GEN_HMAC_BY_KT Success\n");
        break;

    case TCCHSM_GEN_SM3_HMAC_BY_KT:
        req = REQ_HSM_GEN_SM3_HMAC_BY_KT;
        obj_id = 0;
        src_size = sizeof(plain_data);
        key_index = 1;
        mac_size = 32;
        ret = hsm_gen_mac_by_kt(req, obj_id, key_index, plain_data, src_size, mac,
mac_size);

        if (ret) {
            ELOG("hsm_gen_sm3_hmac test fail\n");
            return TCCHSM_ERR;
        }
        if (memcmp(sm3_hmac_out, mac, mac_size) != 0) {
            HexDump(sm3_hmac_out, mac_size);
            HexDump(mac, mac_size);
            ELOG("Wrong cipher data\n");
            return TCCHSM_ERR;
        }
        DLOG("TCCHSM_GEN_SM3_HMAC_BY_KT Success\n");
        break;

    default:
        ELOG("Wrong cmd type\n");
        break;
}

return ret;
}

```

5.6 Generate Digest by HASH

Generate digest by SHA/SM3.

Function

```
int32 hsm_gen_hash(
    uint32 req, uint32 obj_id, uint8 *src_addr, uint32 src_size, uint8 *dig, uint32
    dig_size)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- obj_id: [in] Object ID to distinguish SHA/SM3 functions
- src_addr: [in] Buffer pointer of input data
- src_size: [in] Size of src
- dig: [out] Buffer pointer of result digest
- dig_size: [in] Size of digest

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_gen_hash_test(uint8 cmd)
{
    uint32_t obj_id = 0, src_size = 0, dig_size = 0;
    uint32_t req = 0;
    uint8_t dig[32] = {0};
    int32 ret = TCCHSM_ERR;

    switch (cmd) {
    case TCCHSM_GEN_SHA:
        req = REQ_HSM_GEN_SHA;
        obj_id = OID_SHA1_160;
        dig_size = 20;
        src_size = sizeof(Hash_Message);
        ret = hsm_gen_hash(req, obj_id, Hash_Message, src_size, dig, dig_size);
        if (ret) {
            ELOG("hsm_gen_hash test fail(%d)\n", ret);
            return TCCHSM_ERR;
        }
        if (memcmp(Digest_SHA1_160, dig, dig_size) != 0) {
            HexDump(Digest_SHA1_160, dig_size);
            HexDump(dig, dig_size);
            ELOG("Wrong cipher data\n");
            return TCCHSM_ERR;
        }
        DLOG("TCCHSM_GEN_SHA Success\n");
        break;

    case TCCHSM_GEN_SM3:
        req = REQ_HSM_GEN_SM3;
        obj_id = OID_SM3_256;
        dig_size = 32;
        src_size = sizeof(Hash_Message);
        ret = hsm_gen_hash(req, obj_id, Hash_Message, src_size, dig, dig_size);
        if (ret) {
            ELOG("hsm_gen_hash test fail(%d)\n", ret);
        }
    }
```

```

        return TCCHSM_ERR;
    }
    if (memcmp(Digest_SM3_256, dig, dig_size) != 0) {
        HexDump(Digest_SM3_256, dig_size);
        HexDump(dig, dig_size);
        ELOG("Wrong cipher data\n");
        return TCCHSM_ERR;
    }
    DLOG("TCCHSM_GEN_SHA Success\n");
    break;

default:
    ELOG("Wrong cmd type\n");
    break;
}

return ret;
}

```

5.7 Generate Signature by ECDSA

Generate signature by ECDSA.

Function

```

int32 hsm_run_ecdsa(
    uint32 req, uint32 obj_id, uint8 *key, uint32 key_size, uint8 *dig, uint32 dig_size,
    uint8 *sig, uint32 sig_size)

```

Parameter

■ req:	[in] Refer to the definition of request types in Chapter 6.1.
■ obj_id:	[in] Object ID to distinguish ECDSA algorithm
■ key:	[in] Private key
■ key_size:	[in] Size of private key
■ digest:	[in] Buffer pointer of digest
■ dig_size:	[in] Size of digest
■ sig:	[out] Buffer pointer of ECDSA signature
■ sig_size:	[in] Size of signature

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```

static int32 hsm_run_ecdsa_test(uint8 cmd)
{
    uint32_t obj_id = 0, sig_size = 0, key_size = 0, dig_size = 0;
    uint32_t req = 0;
    uint8_t sig[64] = {0};
    uint8_t pri_key[32] = {0};
    int32 ret = TCCHSM_ERR;

    /* Verify */
    req = REQ_HSM_RUN_ECDSA_VERIFY;
    obj_id = OID_ECC_BP256;
    key_size = sizeof(gzu08ECC_A_Pub);
    dig_size = sizeof(gzu08Digest);
}

```

```

        sig_size = sizeof(Sign);
        ret =
            hsm_run_ecdsa(req, obj_id, gzu08ECC_A_Pub, key_size, gzu08Digest, dig_size,
Sign, sig_size);
        if (ret) {
            ELOG("hsm_run_ecdsa test fail(%d)\n", ret);
            return TCCHSM_ERR;
        }
        DLOG("ecdsa verify success\n");

        /* Signing */
        req = REQ_HSM_RUN_ECDSA_SIGN;
        obj_id = OID_ECC_BP256;
        memcpy(pri_key, gzu08RSA_N, sizeof(pri_key));
        pri_key[0] &= 0x7F;
        key_size = sizeof(pri_key);
        dig_size = sizeof(gzu08Digest);
        sig_size = 64;
        ret = hsm_run_ecdsa(req, obj_id, pri_key, key_size, gzu08Digest, dig_size, sig,
sig_size);
        if (ret) {
            ELOG("hsm_run_ecdsa verify test fail(%d)\n", ret);
            return ret;
        }
        /* Verify */
        req = REQ_HSM_RUN_ECDSA_VERIFY;
        obj_id = OID_ECC_BP256;
        key_size = sizeof(gzu08ECC_A_Pub);
        dig_size = sizeof(gzu08Digest);
        sig_size = sizeof(sig);
        ret =
            hsm_run_ecdsa(req, obj_id, gzu08ECC_A_Pub, key_size, gzu08Digest, dig_size,
sig, sig_size);
        if (ret) {
            ELOG("hsm_run_ecdsa verify test fail\n");
            return ret;
        }
        DLOG("ecdsa signing success\n");

        return ret;
    }

```

5.8 Verify Signature by ECDSA

Verify signature by ECDSA.

Function

```
int32 hsm_run_ecdsa(  
    uint32 req, uint32 obj_id, uint8 *key, uint32 key_size, uint8 *dig, uint32 dig_size,  
    uint8 *sig, uint32 sig_size)
```

Parameter

■ req:	[in] Refer to the definition of request types in Chapter 6.1.
■ obj_id:	[in] Object ID to distinguish ECDSA algorithm
■ key:	[in] Private key
■ key_size:	[in] Size of private key
■ digest:	[in] Buffer pointer of digest
■ dig_size:	[in] Size of digest
■ sig:	[in] Buffer pointer of ECDSA signature
■ sig_size:	[in] Size of signature

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

Refer to the example in Chapter 5.7.

5.9 Generate Signature by RSASSA_PKCS

Generate signature by RSASSA_PKCS.

Function

```
int32 hsm_run_rsa(
    uint32 req, uint32 obj_id, uint8 *modN, uint32 modN_size, uint8 *key, uint32 key_size,
    uint8 *dig, uint32 dig_size, uint8 *sig, uint32 sig_size)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- obj_id: [in] Object ID to distinguish RSA algorithm
- modN: [in] Modulus N for RSA
- modN_size: [in] Size of modN
- key: [in] Private key
- key_size: [in] Size of private key
- digest: [in] Buffer pointer of digest
- digest_size: [in] Size of digest
- sig: [out] Buffer pointer of RSA signature
- sig_size: [in] Size of signature

Return

- 0: TCCHSM_OK
- Others: Refer to TCCTHSM_ERROR definition in chapter 6.2

Example

```
static int32 hsm_run_rsa_test(uint8 cmd)
{
    int32 ret = TCCHSM_ERR;
    uint32 obj_id = 0, req = 0;
    uint32 prikey_size = 0, dig_size = 0, sig_size = 0, key_size = 0, modN_size = 0;
    uint8 sig[128] = {0};

    switch (cmd) {
        case TCCHSM_RUN_RSASSA_PKCS_SIGN:
        case TCCHSM_RUN_RSASSA_PKCS_VERIFY:
            /* Signing */
            req = REQ_HSM_RUN_RSASSA_PKCS_SIGN;
            obj_id = 0;
            modN_size = sizeof(modN);
            prikey_size = sizeof(rsa_prikey);
            dig_size = sizeof(digest);
            sig_size = sizeof(sig);
            ret = hsm_run_rsa(
                req, obj_id, modN, modN_size, rsa_prikey, prikey_size, digest,
                dig_size, sig, sig_size);
            if (ret) {
                ELOG("hsm_run_rsa signing test fail(%d)\n", ret);
                return ret;
            }

            /* Verify */
            req = REQ_HSM_RUN_RSASSA_PKCS_VERIFY;
            obj_id = OID_ECC_BP256;
            key_size = sizeof(rsa_pubkey);
            dig_size = sizeof(digest);
            sig_size = sizeof(sig);
```

```

        ret = hsm_run_rsa(
            req, obj_id, modN, modN_size, rsa_pubkey, key_size, digest, dig_size,
sig, sig_size);
        if (ret) {
            ELOG("hsm_run_ecdsa verify test fail(%d)\n", ret);
            return ret;
        }

        DLOG("rsassa pkcs signing and verifying success\n");
        break;

    case TCCHSM_RUN_RSASSA_PSS_SIGN:
    case TCCHSM_RUN_RSASSA_PSS_VERIFY:
        req = REQ_HSM_RUN_RSASSA_PSS_SIGN;
        obj_id = RSASSA_PSS_OID_HASH;
        modN_size = sizeof(modN);
        prikey_size = sizeof(rsa_prikey);
        dig_size = sizeof(digest);
        sig_size = sizeof(sig);
        ret = hsm_run_rsa(
            req, obj_id, modN, modN_size, rsa_prikey, prikey_size, digest,
dig_size, sig, sig_size);
        if (ret) {
            ELOG("hsm_run_rsa signing test fail(%d)\n", ret);
            return ret;
        }

        /* Verify */
        req = REQ_HSM_RUN_RSASSA_PSS_VERIFY;
        obj_id = RSASSA_PSS_OID_HASH;
        key_size = sizeof(rsa_pubkey);
        dig_size = sizeof(digest);
        sig_size = sizeof(sig);
        ret = hsm_run_rsa(
            req, obj_id, modN, modN_size, rsa_pubkey, key_size, digest, dig_size,
sig, sig_size);
        if (ret) {
            ELOG("hsm_run_ecdsa verify test fail(%d)\n", ret);
            return ret;
        }

        DLOG("rsassa pss signing and verifying success\n");
        break;
    default:
        ELOG("Invalide cmd\n");
        return ret;
        break;
    }

    return ret;
}

```

5.10 Verify Signature by RSASSA_PKCS

Verify signature by RSASSA_PKCS.

Function

```
int32 hsm_run_rsa(
    uint32 req, uint32 obj_id, uint8 *modN, uint32 modN_size, uint8 *key, uint32 key_size,
    uint8 *dig, uint32 dig_size, uint8 *sig, uint32 sig_size)
```

Parameter

- | | |
|----------------|---|
| ■ req: | [in] Refer to the definition of request types in Chapter 6.1. |
| ■ obj_id: | [in] Object ID to distinguish RSA algorithm |
| ■ modN: | [in] Modulus N for RSA |
| ■ modN_size: | [in] Size of modN |
| ■ key: | [in] Private key |
| ■ key_size: | [in] Size of private key |
| ■ digest: | [in] Buffer pointer of digest |
| ■ digest_size: | [in] Size of digest |
| ■ sig: | [out] Buffer pointer of RSA signature |
| ■ sig_size: | [in] Size of signature |

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

Refer to the example in Chapter 5.9.

5.11 Generate Signature by RSASSA_PSS

Generate signature by RSASSA_PSS.

Function

```
int32 hsm_run_rsa(
    uint32 req, uint32 obj_id, uint8 *modN, uint32 modN_size, uint8 *key, uint32 key_size,
    uint8 *dig, uint32 dig_size, uint8 *sig, uint32 sig_size)
```

Parameter

- | | |
|----------------|---|
| ■ req: | [in] Refer to the definition of request types in Chapter 6.1. |
| ■ obj_id: | [in] Object ID to distinguish RSA algorithm |
| ■ modN: | [in] Modulus N for RSA |
| ■ modN_size: | [in] Size of modN |
| ■ key: | [in] Private key |
| ■ key_size: | [in] Size of private key |
| ■ digest: | [in] Buffer pointer of digest |
| ■ digest_size: | [in] Size of digest |
| ■ sig: | [out] Buffer pointer of RSA signature |
| ■ sig_size: | [in] Size of signature |

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

Refer to the example in Chapter 5.9.

5.12 Verify Signature by RSASSA_PSS

Verify signature by RSASSA_PSS.

Function

```
int32 hsm_run_rsa(
    uint32 req, uint32 obj_id, uint8 *modN, uint32 modN_size, uint8 *key, uint32 key_size,
    uint8 *dig, uint32 dig_size, uint8 *sig, uint32 sig_size)
```

Parameter

- | | |
|----------------|---|
| ■ req: | [in] Refer to the definition of request types in Chapter 6.1. |
| ■ obj_id: | [in] Object ID to distinguish RSA algorithm |
| ■ modN: | [in] Modulus N for RSA |
| ■ modN_size: | [in] Size of modN |
| ■ key: | [in] Private key |
| ■ key_size: | [in] Size of private key |
| ■ digest: | [in] Buffer pointer of digest |
| ■ digest_size: | [in] Size of digest |
| ■ sig: | [in] Buffer pointer of RSA signature |
| ■ sig_size: | [in] Size of signature |

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

Refer to the example in Chapter 5.9.

5.13 Get Random Number

Get random number through hardware TRNG block in HSM.

Function

```
int32 hsm_get_rand(uint32 req, uint32 *rng, uint32 rng_size)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- rng: [out] Buffer pointer to get random number
- rng_size: [in] Size of random number. Maximum size is 32 bytes.

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_get_rng_test(uint8 cmd)
{
    uint32_t rng_size = 0, req = 0;
    int32_t ret = TCCHSM_ERR;
    uint32_t rand_num[4] = {0};

    req = REQ_HSM_GET_RNG;
    rng_size = sizeof(rand_num);
    ret = hsm_get_rand(req, (uint32 *)rand_num, rng_size);
    if (ret) {
        ELOG("hsm_get_rand test fail(%d)\n", ret);
        return ret;
    }

    DLOG("%x %x %x %x\n", rand_num[0], rand_num[1], rand_num[2], rand_num[3]);

    return ret;
}
```

5.14 Write OTP

Write the value in reserved HSM area in OTP.

Function

```
int32 hsm_write(uint32 req, uint32 addr, uint8 *buf, uint32 buf_size)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- addr: [in] OTP address to be programmed. Address starts from 0 to 0x1A00, and the size is 1024 bytes.
- data: [in] Buffer pointer of data to program OTP
- data_size: [in] Size of data

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_write_otp_test(uint8 cmd)
{
    uint32 addr = 0, data_size = 0;
    int32 ret = TCCHSM_ERR;

    // write aes test key
    addr = 0x00000030;
    data_size = sizeof(key);

    ret = hsm_write(REQ_HSM_WRITE_OTP, addr, key, data_size);
    if (ret) {
        ELOG("hsm_write_test fail(%d)\n", ret);
        return ret;
    }

    // write mac test key
    addr = 0x00000040;
    data_size = sizeof(mac_key);

    ret = hsm_write(REQ_HSM_WRITE_OTP, addr, mac_key, data_size);
    if (ret) {
        ELOG("hsm_write_test fail(%d)\n", ret);
        return ret;
    }

    DLOG("hsm_write_otp(addr=0x%x) Success\n", addr);
    return ret;
}
```

5.15 Get Firmware Version

Get the HSM firmware version.

Function

```
int32 hsm_get_version(uint32 req, uint32 *x, uint32 *y, uint32 *z)
```

Parameter

- req: [in] Refer to the definition of request types in Chapter 6.1.
- x: [out] Major version
- y: [out] Minor version1
- z: [out] Minor version2

Return

- 0: TCCHSM_OK
- Others: Refer to the definition of TCCTHSM_ERROR in Chapter 6.2.

Example

```
static int32 hsm_get_version_test(uint8 cmd)
{
    int32 ret = TCCHSM_ERR;
    uint32 req = 0, x = 0, y = 0, z = 0;

    req = REQ_HSM_GET_VER;
    ret = hsm_get_version(req, &x, &y, &z);
    if (ret) {
        ELOG("hsm_get_version Err(%d)\n", ret);
        return ret;
    }

    return ret;
}
```

6 APPENDIX

6.1 Request Type

Request Type	Value
REQ_HSM_RUN_AES	0x10010000
REQ_HSM_RUN_AES_BY_KT	0x10020000
REQ_HSM_RUN_SM4	0x10030000
REQ_HSM_RUN_SM4_BY_KT	0x10040000
REQ_HSM_GEN_CMIC	0x10110000
REQ_HSM_GEN_CMIC_BY_KT	0x10120000
REQ_HSM_GEN_GMAC	0x10130000
REQ_HSM_GEN_GMAC_BY_KT	0x10140000
REQ_HSM_GEN_HMAC	0x10150000
REQ_HSM_GEN_HMAC_BY_KT	0x10160000
REQ_HSM_GEN_SM3_HMAC	0x10170000
REQ_HSM_GEN_SM3_HMAC_BY_KT	0x10180000
REQ_HSM_GEN_SHA	0x10210000
REQ_HSM_GEN_SM3	0x10220000
REQ_HSM_RUN_ECDSA_SIGN	0x10310000
REQ_HSM_RUN_ECDSA_VERIFY	0x10320000
REQ_HSM_RUN_RSASSA_PKCS_SIGN	0x10550000
REQ_HSM_RUN_RSASSA_PKCS_VERIFY	0x10560000
REQ_HSM_RUN_RSASSA_PSS_SIGN	0x10570000
REQ_HSM_RUN_RSASSA_PSS_VERIFY	0x10580000
REQ_HSM_GET_RNG	0x10610000
REQ_HSM_WRITE_OTP	0x10710000
REQ_HSM_WRITE_SNOR	0x10720000
REQ_HSM_SET_KEY_FROM_OTP	0x10810000
REQ_HSM_SET_KEY_FROM_SNOR	0x10820000
REQ_HSM_GET_VER	0x20010000

6.2 TCC HSM Error Code

```
typedef enum _TCCHSM_ERROR {
    TCCHSM_OK = 0,
    TCCHSM_ERR = -1,
    TCCHSM_ERR_INVALID_PARAM = -2,
    TCCHSM_ERR_INVALID_STATE = -3,
    TCCHSM_ERR_INVALID_MEMORY = -4,
    TCCHSM_ERR_UNSUPPORT_FUNC = -5,
    TCCHSM_ERR_SOTB_CIPHER = -6,
    TCCHSM_ERR_OCCUPIED_RESOURCE = -7,
} TCCHSM_ERROR;
```

7 REFERENCES

- [1] TCC805x Automotive Security-Specification for HSM

8 REVISION HISTORY

Rev. 1.10: 2020-08-05

- Updated
 - Chapter 3.1.1: Description
 - Chapter 4: Chapter title
 - Chapter 4.1 – Chapter 4.15: Description about argument and example
- Added
 - Chapter 5 HSM to Use HSM APIs on Cortex-R5
 - Chapter 6 Appendix

Rev. 1.00: 2020-07-07

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."